What is a Kubernetes Worker Node?

Answer:
A Kubernetes Worker Node is a machine responsible for running application workloads in the cluster.
It hosts Pods, which contain one or more containers.
The worker node communicates with the Control Plane to receive instructions.
It ensures containers are running in the desired state.
Without worker nodes, Kubernetes cannot run applications.

What components run on a Kubernetes Worker Node?

Answer:
A worker node runs **kubelet**, **container runtime**, and **kube-proxy**.
Kubelet manages Pods and communicates with the API Server.
The **container runtime** runs containers inside Pods.
Kube-proxy handles networking and service routing.
These components together ensure application execution and networking.

What is the role of a worker node in Kubernetes?

Worker nodes are responsible for running application workloads.
They host pods that contain one or more containers.
Each worker node runs kubelet, kube-proxy, and a container runtime.
Worker nodes execute workloads assigned by the scheduler.
They continuously report health and status to the control plane.

What is the role of kubelet in a worker node?

Answer:
Kubelet is the primary agent running on each worker node.
It watches Pod specifications from the API Server.
Kubelet ensures containers are started, stopped, and restarted as required.
It reports node and Pod status back to the Control Plane.
If kubelet fails, Pods on that node become unmanaged.

How does kubelet communicate with the Control Plane?

Answer:

Kubelet communicates with the API Server using HTTPS.
It authenticates using certificates or tokens.
It continuously sends node health and Pod status updates.
It fetches Pod specifications assigned to the node.
This communication is essential for cluster state consistency.

What is a container runtime and why is it needed?

Answer:

The container runtime is responsible for running containers.
Examples include containerd, CRI-O, and Docker (deprecated).
Kubelet does not run containers directly; it uses the runtime.
The runtime pulls images, creates containers, and manages their lifecycle.
Without a container runtime, Pods cannot run.

What is kube-proxy and what problem does it solve?

Answer:

Kube-proxy manages network rules on each worker node.
It enables communication between Pods and Services.
It uses iptables or IPVS to route traffic.
Kube-proxy ensures Service load balancing works correctly.
Without kube-proxy, Services would not function.

How does networking work inside a worker node?

Answer:

Each Pod gets its own IP address.
Containers within a Pod share the same network namespace.
Pod-to-Pod communication works without NAT.
Kube-proxy ensures Service traffic reaches the correct Pod.
Networking is implemented using a CNI plugin.

What is a CNI plugin and why is it required?

Answer:

CNI stands for Container Network Interface.
It is responsible for assigning IP addresses to Pods.
Popular CNI plugins include Calico, Flannel, and Cilium.
Kubernetes relies on CNI for networking, not built-in networking.
Without CNI, Pods cannot communicate.

What happens if a worker node goes down?

Answer:

The Control Plane marks the node as NotReady.
Pods running on that node become unavailable.
The Scheduler reschedules Pods to healthy nodes.
This ensures high availability of applications.
Stateful workloads may require additional recovery steps.

How does Kubernetes schedule Pods to worker nodes?

Answer:

The Scheduler selects a suitable worker node.
It considers CPU, memory, taints, tolerations, and affinity rules.
Once selected, the Pod is assigned to the node.
Kubelet on that node starts the Pod.
Scheduling decisions are made only by the Control Plane.

What are node labels and why are they used?

Answer:

Node labels are key-value pairs attached to worker nodes.
They help control Pod placement.
Schedulers use labels with node selectors or affinity rules.
They are commonly used for environment or hardware segregation.
Labels make scheduling more flexible and controlled.

What are taints and tolerations in worker nodes?

Answer:
Taints repel Pods from a node.
Tolerations allow Pods to be scheduled on tainted nodes.
They are used to protect special-purpose nodes.
For example, dedicated GPU or database nodes.
This ensures workload isolation.

How does kubelet perform health checks?

Answer:
Kubelet monitors container health using probes.
It supports liveness, readiness, and startup probes.
If a liveness probe fails, kubelet restarts the container.
Readiness probe failure removes the Pod from Service endpoints.
This maintains application stability.

What is node resource management?

Answer:
Worker nodes manage CPU and memory resources.
Kubelet enforces resource requests and limits.
It uses cgroups to isolate container resources.
If resources are exhausted, Pods may be evicted.
Proper sizing prevents node instability.

Can worker nodes communicate with each other directly?

Answer:
Yes, worker nodes communicate via the cluster network.
Pods on different nodes can talk using Pod IPs.
This is enabled by the CNI plugin.
There is no NAT between Pods.
This flat network model simplifies microservices communication.

Difference between Control Plane and Worker Node?

Answer:
Control Plane manages cluster state and decisions.
Worker nodes execute application workloads.
Control Plane does not run user containers.
Worker nodes do not make scheduling decisions.
Both are essential for a functioning cluster.

How do logs and metrics work on worker nodes?

Answer:
Containers write logs to stdout and stderr.
Kubelet makes logs accessible via kubectl logs.
Metrics are collected using tools like Metrics Server.
Node-level metrics include CPU and memory usage.
These are critical for monitoring and autoscaling.

What security components run on worker nodes?

Answer:
Kubelet enforces Pod security settings.
Container runtime applies container isolation.
Network policies control Pod traffic.
RBAC controls API access, not node access.
Worker node security is critical to cluster safety.

How does a worker node join a cluster?

Answer:
The node uses a kubeadm join command.
It authenticates using a token and CA certificate.

Kubelet registers the node with the API Server.
Once approved, the node becomes Ready.
It is then eligible for Pod scheduling.

What is the most common worker node failure scenario?

Answer:
Resource exhaustion is the most common issue.
CPU or memory pressure leads to Pod eviction.
Disk pressure can stop container creation.
Network plugin failures cause Pod communication issues.
Monitoring and capacity planning prevent most failures.

Kubernetes Control Plane vs Worker Node (Comparison Table)

==========

Aspect	**Control Plane**	**Worker Node**
Primary Role	Manages the Kubernetes cluster	Runs application workloads
Responsibility	Cluster state, scheduling, decision-making	Pod execution and container runtime
Runs User Workloads	No	Yes
Main Purpose	Brain of the cluster	Muscle of the cluster
Core Components	API Server, Scheduler, Controller Manager, etcd	kubelet, kube-proxy, container runtime
API Interaction	Exposes Kubernetes API	Communicates with API Server
Scheduling	Decides **where** Pods run	Executes assigned Pods

Aspect	**Control Plane**	**Worker Node**
Container Execution	Does not run containers	Runs containers
Cluster State Storage	etcd stores desired & actual state	No persistent cluster state
Failure Impact	Cluster becomes unmanaged	Only Pods on that node affected
High Availability	Critical, must be HA	Optional but recommended
Scaling Impact	Limits cluster operations	Limits workload capacity
Networking Role	Controls networking objects	Handles Pod networking
Security Role	Authentication, Authorization, Admission	Enforces Pod-level security
Resource Management	Global resource awareness	Local CPU & memory enforcement
Auto-healing	Initiates rescheduling	Restarts containers locally
Logging & Metrics	Control-plane metrics	Node & Pod metrics
Join Process	Already part of cluster	Joins using kubeadm join
Cloud Cost	Higher (critical services)	Scales with workload
Interview Analogy	**Manager / Brain**	**Worker / Executor**